Phasing Through the Flames: Rapid Motion Planning with the AGHF PDE for Arbitrary Objective Functions and Constraints

Author Names Omitted for Anonymous Review. Paper-ID [523]

Fig. 1: This paper introduces BLAZE, a Phase 1 - Phase 2 Affine Geometric Heat Flow (AGHF) computational framework, to rapidly solve optimal control problems while respecting robot constraints and avoiding obstacles. It begins with an initial trajectory (shown in **orange** with the color gradient illustrating the evolution in time starting from darkest and going to lightest) that may violate constraints (e.g., the second and fourth pose of the arm are in collision with the boxes and outlined in **red**). If the initial trajectory is infeasible, BLAZE enters Phase 1, where it evolves the trajectory into a trajectory that satisfies all constraints (e.g., in the **blue** trajectory, the Kinova arm has been moved out of collision with the boxes). Once the trajectory satisfies all constraints, Phase 2 begins, optimizing the motion to minimize a user-specified cost function while maintaining feasibility (optimized trajectory shown **green**). In this case, BLAZE optimizes the trajectory to reach a target configuration while avoiding the obstacles while considering the full dynamical model of the arm. Note that optimal control (including Phase 1 and Phase 2) for this 14 dimensional state space model is completed within $3s$ while satisfying input, state, and collision avoidance constraints.

***Abstract*—The generation of optimal trajectories for high-dimensional robotic systems under constraints remains computationally challenging due to the need to simultaneously satisfy dynamic feasibility, input limits, and task-specific objectives while searching over high-dimensional spaces. Recent approaches using the Affine Geometric Heat Flow (AGHF) Partial Differential Equation (PDE) have demonstrated promising results, generating dynamically feasible trajectories for complex systems like the Digit V3 humanoid within seconds. These methods efficiently solve trajectory optimization problems over a two-dimensional domain by evolving an initial trajectory to minimize control effort. However, these AGHF approaches are limited to a single type of optimal control problem (i.e., minimizing the integral of squared control norms) and typically require initial guesses that satisfy constraints to ensure satisfactory convergence. These limitations restrict the potential utility of the AGHF PDE especially when trying to synthesize trajectories for robotic systems. This paper generalizes the AGHF formulation to accommodate arbitrary cost functions, significantly expanding the classes of trajectories that can be generated. This work also introduces a Phase 1 - Phase 2 Algorithm that enables the use of constraint-violating initial guesses while guaranteeing satisfactory convergence. The effectiveness of the proposed method is demonstrated through comparative evaluations against state-of-the-art techniques across various dynamical systems and challenging trajectory generation problems.**

I. Introduction

Optimal Control is a powerful tool for motion planning and control of advanced robotic systems. For robust deployment in trajectory-based robotics-algorithms [1–6], an optimal control algorithm should be (1) computationally efficient to enable online planning, (2) capable of handling nonlinear dynamics and constraints inherent in real-world tasks, (3) scalable to high-dimensional platforms like humanoids and manipulators, and (4) reliably convergent to feasible solutions despite their initialization. However, balancing these criteria is challenging. Current methods to address these challenges can be grouped into two primary approaches – Dynamic Programming (DP) and Variational methods. DP leverages Bellman's Principle of Optimality to derive value functions and optimal policies. In particular, solving the Hamilton-Jacobi-Bellman (HJB) Partial Differential Equation (PDE) offers global optimality guarantees, but necessitates discretizing the entire state space, making it computationally unfeasible for systems beyond five dimensions. Differential Dynamic Programming (DDP) methods, such as Crocoddyl [7], mitigate these issues by applying DP techniques locally along a given

trajectory. Through iterative forward-backward passes, DDP variants achieve more rapid convergence, providing a practical compromise between optimality and computational tractability for high-dimensional robotic systems. Although DDP methods are more computationally efficient than global approaches, they can exhibit poor convergence when initialized far from a local optimum.

Variational methods derive necessary optimality conditions via calculus of variations, offering an alternative route to solving control problems. Within this framework, direct methods discretize a continuous optimal control problem into a nonlinear program (NLP). For example, direct collocation methods [8, 9] approximate trajectories using polynomial functions. Although effective at handling complex constraints, these methods can be computationally intensive for high-dimensional systems and sensitive to discretization choices and initial guesses.

Recently, another PDE-based method using the Affine Geometric Heat Flow Equation has been proposed [10, 11]. Notably these methods have been shown to be able to generate trajectories for high dimensional systems faster than existing methods (e.g., on the order of seconds for systems with more than a 40 dimensional state space model). These methods pose the trajectory optimization problem as the solution to a PDE that evolves an initial trajectory that may not be dynamically feasible into a final trajectory that is dynamically feasible while minimizing control input magnitudes. In contrast to the HJB PDE, the AGHF solution is defined over a two-dimensional domain irrespective of the system's dimension. This enables the AGHF PDE to achieve significant computational speedups while preserving dynamic feasibility in motion planning and incorporating various kinds of constraints. Though these AGHF methods show promise for rapidly generating trajectories for high-dimensional systems, currently they have been restricted to considering only one kind of cost function – the integral of the squared norm of the control inputs along the trajectory. Additionally, to find solutions rapidly these methods often require an initial guess that does not violate any constraint other than the dynamic feasibility constraint.

To address these limitations, this paper proposes BLAZE, a method that builds a generalized AGHF formulation that accommodates arbitrary cost functions, enabling the generation of diverse trajectories that were previously unattainable. This method enables one to rapidly compute dynamically feasible trajectories for complex, highly dynamic tasks for high dimensional systems by leveraging spatial vector algebra, a pseudospectral method and a Phase 1-Phase 2 algorithm for the solving the AGHF PDE (as illustrated in Figure 1). The contributions of this paper are three-fold: First, this paper illustrates how to formulate the AGHF Action Functional to solve optimal control problems with arbitrary cost functions. Second, this paper describes how to implement a Phase 1-Phase 2 style method that enables the AGHF initial guess to violate the constraints while still generating feasible solutions rapidly. Third, this paper describes how to enforce input constraints in the AGHF. The utility of these contributions is illustrated by comparing the performance of the proposed method to state of the art trajectory optimization techniques and a hardware demonstration of the proposed method on the Kinova Gen3 robot.

The remainder of the paper is arranged as follows: Section II presents the background and introduces the relevant notation for the paper. Section III introduces the AGHF and discusses the underlying theory associated with the AGHF. Section IV details how to incorporate constraints into the AGHF to enable actions like obstacle avoidance.

II. Preliminaries

This section introduces the notation used throughout this manuscript. This paper is focused on performing trajectory optimization for robot systems whose dynamics can be written as follows:

$$H(q(t))\ddot{q}(t) + C(q(t), \dot{q}(t)) = Bu(t), \tag{1}$$

where $q(t) \in \mathbb{R}^N$ is the configuration of the robot at time t, $u(t) \in \mathbb{R}^m$ is the input applied to the robot at time t, $H(q(t))$ is the mass matrix, $C(q(t), \dot{q}(t))$ is the grouped Coriolis and gravity term and B is the actuation matrix. For convenience, let $\mathtt{x}(t)$ correspond to the vector of $q(t)$ and $\dot{q}(t)$. To be consistent with the notation in the rest of the paper we refer to the first N and last N components of $\mathtt{x}(t)$ as $\mathtt{x}_{P1}(t)$ and $\mathtt{x}_{P2}(t)$, respectively. Additionally, let $\mathbf{0}$ be a $N \times 1$ vector of zeros. Using these definitions, we can represent the dynamics of the robot (1) as a control affine system:

$$\dot{\mathtt{x}}(t) = F_d(\mathtt{x}(t)) + F(\mathtt{x}(t))u(t), \tag{2}$$

where

$$F_d(\mathtt{x}(t)) = \begin{bmatrix} \mathtt{x}_{P2}(t) \\ -H^{-1}(\mathtt{x}_{P1}(t))C(\mathtt{x}_{P1}(t), \mathtt{x}_{P2}(t)) \end{bmatrix} \tag{3}$$

$$F(\mathtt{x}(t)) = \begin{bmatrix} 0_{N\times m} \\ H^{-1}(\mathtt{x}_{P1}(t))B \end{bmatrix} \tag{4}$$

For convenience, we assume without loss of generality that we are interested in the evolution of the system for $t \in [0, T]$. Lastly, let L^2 refer to the set of square integrable functions defined for $t \in [0, T]$. Throughout this paper, we also utilize the following definition that is used throughout the calculus of variations because it allows one to describe how functions behave as they go to infinity:

Definition 1 (Coercive Functions [12, Section 8.2])**.** *A function $f : \mathbb{R}^n \to \mathbb{R}$ is called* coercive *if there exist constants $\alpha > 0, \beta \geq 0$ such that $f(x) \geq \alpha\|x\|_\infty - \beta$ for all x.*

The objective of this paper is to develop an algorithm to construct a trajectory beginning from some user-specified initial condition, $\mathtt{x}_0$, and ending in some user-specified terminal condition, $\mathtt{x}_f$, while satisfying state and input constraints and the dynamics in (2) for all $t \in [0, T]$ all while minimizing an arbitrary cost function. For convenience, let the zero superlevel set of functions g_j for each $j \in \mathcal{J}$ and h_i for each $i \in \mathcal{I}$ represent a collection of state and input inequality constraints

where $\mathcal{J} \subset \mathbb{N}$ and $\mathcal{I} \subset \mathbb{N}$ are each finite sets. Using these definitions, one can formulate the trajectory optimization problem:

$$\begin{aligned}
\inf_{u \in L^2} \quad & \int_0^T c(\mathtt{x}(t), \dot{\mathtt{x}}(t), u(t))\, dt && \text{(OCP)} \\
\text{s.t.} \quad & \dot{\mathtt{x}}(t) = F_d(\mathtt{x}(t)) + F(\mathtt{x}(t))u(t), && \forall t \in [0,T], \\
& g_j(\mathtt{x}(t), \dot{\mathtt{x}}(t)) \leq 0 && \forall t \in [0,T], \forall j \in \mathcal{J}, \\
& h_i(u(t)) \leq 0 && \forall t \in [0,T], \forall i \in \mathcal{I}, \\
& \mathtt{x}(0) = \mathtt{x}_0, \\
& \mathtt{x}(T) = \mathtt{x}_f,
\end{aligned}$$

The goal of this paper is to develop a method to rapidly generate trajectories for high-dimensional systems while incorporating multiple constraints using the Affine Geometric Heat Flow Partial Differential Equation to solve (OCP). To ensure the convergence of the AGHF, we make the following assumption:

Assumption 2 (Continuity of Dynamics)**.** *Both F_d and F are C^2, globally Lipschitz continuous (with constants Y_1 and Y_2 respectively), and F has constant rank almost everywhere in $\mathbb{R}^n$. Suppose c, g_j, and h_i are C^2 in all of their arguments for all $j \in \mathcal{J}$ and $i \in \mathcal{I}$. Additionally, assume that there exists a trajectory that satisfies the constraints of (OCP).*

Note that the dynamics of rigid-body robotic systems are smooth and Lipschitz continuous when restricted to a compact domain. In addition, we do not assume that the trajectory that satisfies the constraints of (OCP) is given to the user. Rather we only assume its existence.

III. The Affine Geometric Heat Flow (AGHF) Partial Differential Equation

The Affine Geometric Heat Flow (AGHF) Partial Differential Equation (PDE) is a parabolic PDE that attempts to solve (OCP). The AGHF deforms an initial trajectory, which can be any trajectory including one that does not satisfy the dynamics, into a dynamically feasible final trajectory that minimizes some user-specified cost. This section summarizes the background knowledge and theory of the AGHF. A more detailed treatment of the subjects discussed here can be found in [10, 11, 13, 14]. *Throughout this section, we assume that there are no inequality constraints in (OCP). The inequality constraint case is considered in Section IV.*

A. Homotopies and the Action Functional

To describe the evolution of trajectories by the AGHF PDE, we begin by defining a homotopy: $x : [0,T] \times [0, s_{max}] \to \mathbb{R}^{2N}$, that is twice differentiable with respect to its first argument and differentiable with respect to its second argument. For convenience, we denote $x(t,s)$ by $x_s(t)$ and we denote $\frac{\partial x}{\partial t}(t,s)$ by $\dot{x}(t,s)$ or $\dot{x}_s(t)$. As is done in Section II, we refer to the first N and last N components of x_s as x_{P1} and x_{P2}, respectively. Additionally, let the first and second time derivatives of these states be defined as $\dot{x}_{P1}$, $\dot{x}_{P2}$ and $\ddot{x}_{P1}$, $\ddot{x}_{P2}$, respectively.

Next, define the *Lagrangian* $L : \mathbb{R}^{2N} \times \mathbb{R}^{2N} \to \mathbb{R}$, which we assume is C^2 and twice differentiable with respect to any of its arguments. Subsection III-C describes how to select L to ensure that the AGHF minimizes the cost function in (OCP). Finally, define the *Action Functional*:

$$\mathcal{A}(x_s) = \int_0^T L(x_s(t), \dot{x}_s(t))dt. \tag{5}$$

Using these definitions, we can define the AGHF PDE:

Definition 3 (Affine Geometric Heat Flow)**.** *The* Affine Geometric Heat Flow *is a parabolic partial differential equation defined as:*

$$\begin{aligned}
\frac{\partial x}{\partial s}(t,s) = M^{-1}(x(t,s))\Bigg(& \frac{d}{dt}\frac{\partial L}{\partial \dot{x}_s}(x_s(t), \dot{x}_s(t)) \\
& - \frac{\partial L}{\partial x_s}(x_s(t), \dot{x}_s(t))\Bigg),
\end{aligned} \tag{6}$$

with the following boundary conditions:

$$\begin{aligned}
x_s(0) &= \mathtt{x}_0, \quad \forall s \in [0, s_{max}], \\
x_s(T) &= \mathtt{x}_f, \quad \forall s \in [0, s_{max}].
\end{aligned} \tag{7}$$

where $M : \mathbb{R}^{2N} \to \mathbb{S}^{2N}_+$ is a user specified matrix valued function and $\mathbb{S}^{2N}_+$ is the set of positive semi-definite matrices.

Note that this AGHF formulation differs from standard approaches in that here we allow $M(x(t,s))$ to be an arbitrary positive semi-definite and invertible matrix. This enables us to consider general cost functions. The traditional AGHF formulation designs M specifically to penalize evolution toward dynamically infeasible states while prioritizing control effort reduction. However, as we show in this paper, for general cost functions, M, should be defined differently.

When solving the AGHF PDE, one begins by specifying an initial curve $x_{init} : [0,T] \to \mathbb{R}^{2N}$. As the AGHF PDE evolves forward in s, one can prove that the action functional is minimized. In addition, if during that evolution the AGHF converges to a curve where the right-hand side of the AGHF PDE is equal to 0, then one has found a curve that extremizes the Action Functional. Such a curve is called a *steady state solution*. We formalize these observations in the following Lemma that describes the convergence properties of the AGHF and whose proof can be found in Appendix A.

Lemma 4 (Action Functional Along the AGHF Homotopy)**.** *Let x_s satisfy the AGHF PDE* (6)*. Then, $\frac{d\mathcal{A}(x_s)}{ds} \leq 0$ for all s. In addition, if the right hand side of the AGHF PDE when evaluated at x_{s^*} is equal to 0 for some $s^* \in [0, s_{max})$, then $\frac{d\mathcal{A}(x_{s^*})}{ds} = 0$.*

In other words, given the Action Functional (5), as the AGHF PDE evolves forward in s, the Action Functional is minimized. In addition, if during that evolution the AGHF converges to a curve where the right hand side of the AGHF PDE is equal to 0, then one has found a *steady state solution* to the AGHF.

B. Solving the AGHF Rapidly

Unlike the Hamilton-Jacobi-Bellman (HJB) PDE, which suffers from the curse of dimensionality due to its exponential scaling with state dimension, the AGHF PDE scales polynomially [15]. This favorable scaling arises because the solution to the AGHF PDE has a two-dimensional domain and and has a range whose dimension grows linearly with the state dimension. Because the AGHF solution has a two-dimensional domain, the AGHF PDE can be solved using the Method of Lines (MOL) [16].

The MOL discretizes the AGHF solution domain along one dimension (typically t) into a set of nodes, reformulating the PDE at each node as a system of coupled Ordinary Differential Equations (ODEs). These coupled ODEs can then be solved in s using numerical ODE solvers. During each ODE solver step, the right-hand side (RHS) of the AGHF PDE must be evaluated at every node. For high-dimensional systems, this process becomes computationally expensive, as it requires repeatedly computing the system dynamics and their derivatives.

In classical MOL AGHF implementations [10], the nodes are uniformly spaced, and the solution accuracy improves as more nodes are introduced. However, because the number of function evaluations scales linearly with the number of nodes, a finer discretization requires frequent evaluations of the AGHF RHS, significantly increasing computational cost. Recent approaches [11] address this by employing a pseudospectral MOL that strategically reduces the number of required nodes while maintaining accuracy. This pseudospectral formulation enables precise computation of time derivatives. Additionally, one can accelerate the evaluation of the AGHF's RHS by utilizing spatial vector algebra and rigid body dynamics algorithms [11].

C. Ensuring $\mathcal{A}$ Coincides with the (OCP) Cost Function

To ensure that the Action Functional, $\mathcal{A}$, being minimized coincides with minimizing an arbitrary cost function, $c(\mathbf{x}(t), \dot{\mathbf{x}}(t), u(t))$, of the (OCP) we must design L carefully. Before defining this action functional we first introduce one additional definition:

Definition 5 (Control Extraction)**.** *Suppose x_s corresponds to a continuously differentiable trajectory at any $s \in [0, s_{max}]$. Let $u_s : [0, T] \to \mathbb{R}^m$ be the* extracted control input *given by:*

$$u_s(t) = \begin{bmatrix} 0_{N\times N} & I_{N\times N} \end{bmatrix} \bar{F}(x_s(t))^{-1}(\dot{x}_s(t) - F_d(x_s(t))). \tag{8}$$

where

$$\bar{F}(x_s(t)) = \begin{bmatrix} F_c(x_s(t)) & F(x_s(t)) \end{bmatrix} \in \mathbb{R}^{2N\times 2N}, \tag{9}$$

and $F_c(x_s(t)) \in \mathbb{R}^{2N\times(2N-m)}$ is a differentiable matrix such that $\bar{F}$ is invertible for all $x_s(t)$ in $\mathbb{R}^{2N}$.

This definition describes how to synthesize a control input given a specific trajectory. For notational convenience, we have omitted the explicit dependency of u_s on x_s and $\dot{x}_s$. Note that F_c in the above definition can be obtained using the Gram-Schmidt procedure. We give an explicit example of how to construct F_c when describing our experimental results in Section VI. Before moving on, we call attention to one important consideration: it remains unclear whether applying this extracted control input to the dynamical system (2) would reproduce the original trajectory x_s used in its creation. This question is answered in Theorem 7.

Given Definition 5, we can define a Lagrangian and associated Action Functional that encode solutions to (OCP) as we describe next:

Definition 6 (Lagrangian and Action Functional for (OCP))**.** *Let $u_s : [0, T] \to \mathbb{R}^m$ be the extracted control input at some $s \in [0, s_{max}]$ using Definition 5. Define L in terms of the extracted control input $u_s(t)$ as:*

$$\begin{aligned} L(x_s(t), \dot{x}_s(t)) = k_d\|\dot{x}_{P1}(s,t) - x_{P2}(s,t)\|_2^2 \\ + c(x_s(t), \dot{x}_s(t), u_s(t)) \end{aligned} \tag{10}$$

where $k_d > 0$. Then the corresponding Action Functional is given by

$$\begin{aligned} \mathcal{A}(x_s) = \int_0^T \Big(k_d\|\dot{x}_{P1}(s,t) - x_{P2}(s,t)\|_2^2 \\ + c(x_s(t), \dot{x}_s(t), u_s(t)) \Big) dt \end{aligned} \tag{11}$$

In summary, for each s, the Action Functional with L as described by Definition 6 consists of the integral of an arbitrary objective function, $c(\mathbf{x}(t), \dot{\mathbf{x}}(t), u(t))$, plus the error between the velocity states, x_{P2}, and the derivative of the position state, $\dot{x}_{P1}$. For trajectories that satisfy the system dynamics (2), this error term vanishes, reducing the Action Functional to the integral of $c(\mathbf{x}(t), \dot{\mathbf{x}}(t), u(t))$. Notably, as Lemma 4 shows, as the AGHF evolves the Action Functional decreases, which coincides with the objective function in (OCP) for dynamically feasible trajectories.

However, it is unclear whether the solution generated by the AGHF PDE eventually satisfies the system dynamics (2). The next result whose proof can be found in Appendix B resolves this concern:

Theorem 7 (AGHF Generates Feasible Trajectories)**.** *Consider a system governed by the dynamics in* (2)*, with an initial state $\mathbf{x}_0$ and desired final state $\mathbf{x}_f$. Suppose Assumption 2 is satisfied, F_c in Definition 5 is a continuous function, and the cost function in the Action Functional* (11) *is coercive. Then there exists C_1, C_2, and $C_3 > 0$, such that for any $k_d > 0$, there exists an open set $\Omega_{k_d} \subset \{x \in C^1([0, T] \to \mathbb{R}^{2N}) \mid x(0) = \mathbf{x}_0, x(T) = \mathbf{x}_f\}$ such that as long as the initial curve to the AGHF PDE satisfies $x_{init} \in \Omega_{k_d}$ then for sufficiently large s_{max} the integrated path $\tilde{x}$ using the extracted control input with $x_{s_{max}}$ plugged into the dynamics* (2) *satisfies the following error bound for any $t \in [0, T]$:*

$$\|\tilde{x}(t) - x_{s_{max}}(t)\|_2 \leq \sqrt{\frac{3TC_1C_2^2}{k_d} \exp\left(3T(Y_1^2T + Y_2^2C_3)\right)} \tag{12}$$

Theorem 7 proves that for sufficiently large k_d and s_{max}, the control input extracted from the solution to the AGHF PDE can be used to generate a trajectory that is arbitrarily close to a dynamically feasible trajectory of (2) and provides an explicit bound on how close the trajectory obtained by applying (8) is to a feasible trajectory of (OCP). With this result, we have an AGHF formulation that tries to minimize the objective function in (OCP) and enables one to generate dynamically feasible trajectories.

Remark 8 (Relationship to Previous Lagrangians)**.** *Note that in Definition 6 if L is replaced by*

$$L(x_s(t), \dot{x}_s(t)) = \big(\dot{x}_s(t) - F_d(x_s(t))\big)^T G(x_s(t)) \cdot \big(\dot{x}_s(t) - F_d(x_s(t))\big), \tag{13}$$

where G is given by

$$G = \begin{bmatrix} kI_{N\times N} & 0_{N\times N} \\ 0_{N\times N} & H^T H \end{bmatrix} \in \mathbb{R}^{2N\times 2N} \tag{14}$$

and $M = G$, then the resulting Action Functional and AGHF correspond to the standard formulation used in the literature [10, 11], which minimizes the squared control input (i.e $c(x_s(t), \dot{x}_s(t), u_s(t))$ is $\|u_s(t)\|_2^2$). However, the formulation in Definition 6 along with Theorem 7 generalizes this framework to accommodate a broader class of optimal control problems.

IV. Incorporating Arbitrary Constraints into the AGHF

The previous section assumed that there were no constraints in (OCP) other than the dynamics constraints. This section discusses how to enforce general state and input constraints.

A. Constrained Lagrangian

We incorporate constraints into the AGHF by using a penalty term in the Lagrangian:

Definition 9 (Constrained Lagrangian)**.** *Let k_{cons} be some large positive real number, and let $g_j(x(t), \dot{x}(t))$ be the j-th inequality constraint evaluated at $x(t)$ and $\dot{x}(t)$. Let the* Constrained Lagrangian *denoted by L_{cons} be defined as:*

$$L_{cons}(x(t), \dot{x}(t)) = L(x(t), \dot{x}(t)) + \sum_{j\in\mathcal{J}} b(g_j(x(t), \dot{x}(t))) \tag{15}$$

where

$$b(g_j(x(t), \dot{x}(t))) = k_{cons} \cdot (g_j(x(t), \dot{x}(t)))^2 \cdot S(g_j(x(t), \dot{x}(t))), \tag{16}$$

where $S : \mathbb{R} \to \mathbb{R}$ is defined as follows:

1) *$S : \mathbb{R} \to \mathbb{R}$ is a positive, differentiable function,*
2) *$S(g_j(x(t), \dot{x}(t))) = 0$ when $g_j(x(t), \dot{x}(t)) \leq 0$ and*
3) *$S(g_j(x(t), \dot{x}(t))) = 1$ when $g_j(x(t), \dot{x}(t)) > 0$.*

The function S ensures that the penalty term is activated only when the AGHF trajectory moves towards constraint violation. An example of such a function is described during the description of the experiments in Section VI. With a sufficiently large k_{cons}, the penalty term steers the trajectory away from infeasible regions, preventing convergence to solutions that violate constraints.

Using the Constrained Lagrangian, one can construct a corresponding AGHF by applying Definition 3:

Lemma 10 (AGHF for a Constrained Lagrangian)**.** *Consider a Constrained Lagrangian, L_{cons}, as in Definition 9. Then the AGHF PDE is given by:*

$$\frac{\partial x}{\partial s} = M^{-1}(x_s(t))\bigg(\frac{d}{dt}\frac{\partial L}{\partial \dot{x}_s}(x_s(t), \dot{x}_s(t)) - \frac{\partial L}{\partial x_s}(x_s(t), \dot{x}_s(t)) + \sum_{j\in\mathcal{J}}\left(\frac{d}{dt}\frac{\partial b(g_j(x_s(t), \dot{x}_s(t)))}{\partial \dot{x}_s(t)} - \frac{\partial b(g_j(x_s(t), \dot{x}_s(t)))}{\partial x_s(t)}\right)\bigg) \tag{17}$$

Notice that L_{cons}, still satisfies the properties of the Lagrangian introduced in Section III-A. As a result, the convergence properties shown in Lemma 4 apply to the Constrained Lagrangian. If the function b in the definition of L_{cons} is coercive, then the results of Theorem 7 also apply to ensure that a dynamically feasible trajectory is found. Note this does not guarantee that the inequality constraints within (OCP) are eventually satisfied. This is particularly an issue when the initial trajectory passed to the homotopy violates the constraints. We address this particular challenge in Section V.

B. Incorporating Input Constraints in the Lagrangian

This section discusses how to rapidly compute the derivatives of the input penalties which are required to enforce input constraints. Before introducing this formulation, we briefly describe how alternative AGHF formulations have enforced input constraints. Existing approaches to enforce input constraints using the AGHF require augmenting the state by treating inputs as additional states, and enforcing the input constraints as state constraints[10]. For high-dimensional robotic systems, this approach substantially increases the dimension of state space of the system and, consequently, the dimension of the AGHF PDE.

To avoid this issue, we instead enforce the input constraints by appending L_{cons} with the term $\sum_{i\in\mathcal{I}} b(h_i(u_s(t))$. However to construct the AGHF as in Lemma 10 for this even larger Constrained Lagrangian, one must be able to compute partial derivatives of $b(h_i(u_s(t))$ with respect to $x_s(t)$ and $\dot{x}_s(t)$. This requires then applying the chain rule and computing partial derivatives of $u_s(t)$ with respect to $x_s(t)$ and $\dot{x}_s(t)$. This can be challenging for high-dimensional robotic systems. Fortunately, one can leverage the following lemma, whose proof can be found in Appendix C:

Lemma 11. *Suppose dynamics are given by* (1) *where $B = I$ then the control extraction formula* (8) *is equivalent to inverse dynamics and u_s can be computed directly using inverse dynamics.*

From Lemma 11 it follows that one can compute u_s using Rigid Body Dynamics Algorithms, such as the Recursive

Newton-Euler Algorithm (RNEA). This can be used to efficiently compute $u_s(t)$ using $x_s(t)$ and $\dot{x}_s(t))$ [17]. By employing an extended version of RNEA that recursively applies the chain rule [18] we can also rapidly evaluate the derivatives of the control inputs $\frac{\partial u_s}{\partial x_s}$ and $\frac{\partial u_s}{\partial \dot{x}_s}$.

V. Designing a Phase 1 -Phase 2 Algorithm using the AGHF

Similar to other trajectory optimization methods, AGHF requires x_{init} as an initial guess. As detailed in Section III, the AGHF PDE transforms this initial trajectory x_{init} into an optimal final trajectory. If x_{init} does not satisfy the optimal control problem's constraints, the AGHF solver must address constraint violations, leading to larger penalties that make the AGHF evaluate to larger and larger values, which can cause the ODE solver to take smaller steps to ensure solution accuracy, which increases the convergence time. Previous approaches [11] mitigate this issue by providing initial guesses that satisfy the constraints, facilitating faster AGHF convergence. However, designing such feasible initial trajectories is challenging for systems with many complex constraints such as input constraints.

To enhance the efficiency of the AGHF and enable rapid convergence, we propose a Phase 1- Phase 2 Algorithm using the AGHF. Phase 1-Phase 2 Methods are used in optimization to first drive an initial guess into a feasible region (Phase 1) before optimizing the objective function while maintaining feasibility (Phase 2) [19]. This approach aids an optimization process in beginning from a well-conditioned, constraint-satisfying state. This results in improved convergence properties and computational efficiency. To implement this within AGHF, we formulate two distinct Action Functionals – one for each phase.

In Phase 1, the goal is to guide the initial trajectory into the feasible set while promoting dynamic feasibility. Leveraging Definition 6, we construct an Action functional that primarily penalizes constraint violations to achieve this, with an additional term to encourage dynamic feasibility. In Phase 2, we apply the generalized AGHF Action Functional with constraints (10) to maintain feasibility while minimizing the objective cost.

We first introduce the form of the Phase 1 Action Functional:

Definition 12. *Let the* Phase 1 Action Functional *for a general state constraint $g_j(x_s(t), \dot{x}_s(t))$ be defined by*

$$\mathcal{A}(x_s) = \int_0^T k_d \|\dot{x}_{P1}(s,t) - x_{P2}(s,t)\|_2^2 + \sum_{j \in \mathcal{J}} b(g_j(x_s(t), \dot{x}_s(t))) \quad (18)$$

and let the corresponding AGHF RHS be expressed as:

$$\frac{\partial x}{\partial s}(t,s) = I_{2N\times 2N} \cdot \left(\frac{d}{dt}\frac{\partial L_d}{\partial \dot{x}_s}(x_s(t), \dot{x}_s(t)) - \frac{\partial L_d}{\partial x_s}(x_s(t), \dot{x}_s(t))\right) + \sum_{j \in \mathcal{J}} \left(\frac{d}{dt}\frac{\partial b(g_j(x_s(t), \dot{x}_s(t)))}{\partial \dot{x}_s(t)} - \frac{\partial b(g_j(x_s(t), \dot{x}_s(t)))}{\partial x_s(t)}\right), \quad (19)$$

where $I_{2N\times 2N}$ is the identity matrix and

$$L_d(x_s(t), \dot{x}_s(t)) = \left(\dot{x}_s(t) - F_d(x_s(t))\right)^T \cdot \begin{bmatrix} k_d I_{N\times N} & 0_{N\times N} \\ 0_{N\times N} & 0_{N\times N} \end{bmatrix} \left(\dot{x}_s(t) - F_d(x_s(t)\right). \quad (20)$$

Note that one can extend this definition to incorporate input constraints $h_i(u_s(t))$. Phase 2 utilizes the AGHF RHS from(10), initializing the trajectory evolution with the solution obtained from Phase1. Given a sufficiently large penalty parameter k_{cons} and evolution time s_{max}, where $k_{cons} > k_d$, the Phase 1 trajectory evolves towards a trajectory that minimizes constraint violation while promoting some dynamic feasibility.

Phase 2 is initialized with the final trajectory from Phase 1 and proceeds by optimizing the objective function while maintaining feasibility. As shown in Section IV-A, the constraint penalty term ensures that for sufficiently large k_{cons} and s_{max}, where $k_{cons} > k_d$ if the trajectory starts in the feasible set, it will remain within the feasible set throughout the evolution. Consequently, as the AGHF PDE progresses, the trajectory is driven toward a minimizer of the specified cost function that is dynamically feasible while satisfying all constraints.

VI. Experiments and Results

This section evaluates the speed and performance of BLAZE on a variety of scenarios and robot platforms. We begin by explaining the experimental setup. Next, we evaluate the scalability of BLAZE when compared to the state of the art trajectory optimization algorithm. Then, we investigate the performance of the methods in scenarios where obstacles are present and then describe the translation of these results onto a real-world platform. We conclude the section by summarizing the different evaluations.

A. Implementation of BLAZE

Our numerical implementation of BLAZE follows the pseudospectral MOL approach adopted in [11] which is summarized in Section III-B. Throughout these experiments, for BLAZE we choose the following activation function $S(g_j(x, \dot{x}))$ for state constraints that satisfies the properties highlighted in Definition 9:

$$S(g_j(x, \dot{x})) = \frac{1}{2} + \frac{1}{2}\tanh(c_{\text{cons}} \cdot g_j(x, \dot{x})) \quad (21)$$

where c_{cons} is a hyper-parameter that determines how fast $S(g_j(x))$ transitions from 0 to 1, once the constraint is violated. Note that we apply the same activation function for the input constraints as well, where in that case the activation

is given by $S(g_j(u))$ instead. Note that the Constrained Lagrangian we choose in Phase 1 of our implementation requires that we satisfy input box-constraints for each of the examples described below. Whereas in Phase 2, we try minimize the square of the control effort. In either instance, note that the Lagrangian satisfies the coercive requirement due to the definition of the extracted control input (Definition 5).

B. Experimental Setup

We evaluate BLAZE and compare it to Crocoddyl [7], Aligator [20], and RAPTOR [21]. Note we compare against these methods because they each are able to perform optimal control while considering the full order dynamics of a robot. We solve a number of fixed time and fixed initial and final state optimization problems. In all the problems, we enforce input constraints and state constraints and in problems with obstacles we also enforce obstacle avoidance constraints. All the experiments were run on an Ubuntu 22.04 machine with an AMD EPYC 7742 64-Core @ 256x 2.25GHz CPU.

1) Selection of Parameters

The solution produced by any of the mentioned trajectory optimization algorithms depends heavily on the selection of initial guess and on the parameters of the solver. This section explains the rationale used to choose the parameters.

For most experiments we use the parameters reported in the grid search results of [11] with the following caveat. When the experiments are qualitatively different from the ones considered in that paper (e.g., novel constraints or types of obstacles), we sequentially vary the parameters until finding parameters those that yield a feasible solution. If the previous procedure fails, we run a small grid search around the parameters reported in [11]. Appendix D summarizes all the parameters chosen for the different methods for all the experiments and explain the meaning of each.

2) Evaluating Success or Failure

Each tested numerical method generates an open-loop control input, $u^* : [0, T] \to \mathbb{R}^m$ and corresponding system trajectory, $x^* : [0, T] \to \mathbb{R}^{2N}$. However, whether the robot can accurately track these computed solutions in practice is inobvious. Thus, to fairly evaluate constraint satisfaction and account for integration errors when applying the open-loop control, we integrate forward using the following feedback controller:

$$u_{fb}(t) = u^*(t) + k_p(x^*_{P1}(t) - q(t)) + k_v(x^*_{P2}(t) - \dot{q}(t)), \quad (22)$$

where $k_p = k_v = 100$ for each experiment, and $q(t)$ and $\dot{q}(t)$ represent the robot's position and velocity at time t, respectively. The system dynamics are then integrated forward using (22) to assess tracking accuracy and constraint adherence.

In obstacle-free experiments, a solution is considered *successful* if the infinity norm of the error between the forward-integrated final state and x_f is below a fixed threshold $\epsilon = 0.05$ and the level of constraint violation is of the forward integrated solution is less than 5% of the maximum constraint bound (e.g. for $u_{max} = 5$, and $u_{min} = -5$ then the control must satisfy $-5.25 \leq u(t) \leq 5.25, \ \forall t \in [0, T]$). In experiments with obstacles, a trial is considered *successful* if it meets the previous criteria and, additionally, the joint positions of the forward integrated solution, sampled at a time resolution of $\Delta t = 10^{-2}$s, remain outside of the obstacles at each joint frame.

3) Aligning Initial Guesses

For all the experiments, each method is given the same initial guess x_{init} to ensure a fair comparison across methods. Because Crocoddyl and Aligator, both DDP-based methods, optimize over control inputs as decision variables, we provide them with an initial guess of $u_{init} : [0, T] \to \mathbb{R}^N$. This is obtained via inverse dynamics using position $q : [0, T] \to \mathbb{R}^N$, velocity $\dot{q} : [0, T] \to \mathbb{R}^N$ and acceleration $\ddot{q} : [0, T] \to \mathbb{R}^N$ extracted from x_{init}. By default x_{init} provides q and $\dot{q}$. To compute the acceleration trajectory $\ddot{q} : [0, T] \to \mathbb{R}^N$, we fit a Chebyshev polynomial to x_{init} and apply the Chebyshev differentiation matrix $D : \mathbb{R}^{p+1} \to \mathbb{R}^{p+1}$ as described in [22, (21.2)]. Specifically, the differentiation matrix D approximates the derivative of the fitted polynomial, allowing the computation of $\ddot{q}$ from the polynomial coefficients. The resulting trajectories q, $\dot{q}$ and $\ddot{q}$ are then utilized in the inverse dynamics computation to generate u_{init}. In contrast, RAPTOR formulates trajectory optimization with Bezier polynomial coefficients as decision variables. To ensure a consistent initial guess, we fit a Bezier polynomial to x_{init}. x_{init} and use the fitted polynomial's coefficients as RAPTOR's initial input.

C. BLAZE Scalability with Increasing System Dimension

This section compares BLAZE, Crocoddyl, Aligator and RAPTOR in solving fixed-time trajectory optimization problems with fixed initial and final states across multiple systems. The goal is to understand how each method's solve time scales with increasing system dimension. In each of these trials input and joint limits are enforced, but there are no obstacle avoidance constraints. The objective function is formulated as minimizing control effort along the trajectory.

We do this evaluation for a 1-, 2-, 3-, 4-, 5-link pendulum model, a 7DOF Kinova Gen3 arm, a double 7DOF Kinova Gen3 arm and a triple 7DOF Kinova Gen3. Figure 2 shows how each of the aforementioned methods scale with increasing N. We see that BLAZE is able to scale more favorably that the other methods and solves faster than all the other methods for most of the evaluated robot systems with $N > 1$. Refer to tables VIII, IV and VII for the specific parameter values used for BLAZE, Aligator and Crocoddyl respectively.

D. Kinova Gen3 Sphere Obstacle Avoidance

This section compares BLAZE and Aligator in solving multiple fixed-time trajectory optimizations to get the Kinova Gen3 from some initial state to some final state while avoiding sphere obstacles. Note, we do not run Crocoddyl or RAPTOR in these experiments, because they did not have any examples doing sphere obstacle avoidance. In these experiments, we design x_{init} such that it starts in collision, to evaluate how effective the proposed method is in moving the initial guess out of constraint violation during Phase 1 before solving Phase

Fig. 2: A bar plot comparing the mean solve times for four different trajectory optimization algorithms: BLAZE, RAPTOR, Crocoddyl and Aligator. For $N \leq 5$ the results correspond to the 1-5 link pendulum, $N = 7$ corresponds to the Kinova Gen3 Arm, $N = 14$ corresponds to a bimanual system of two Kinova Gen3 Arms, $N = 21$ corresponds to a trimanual system of three Kinova Gen3 Arms. Each experiment was run ten times. Overall, we see that BLAZE shows better scalability and solve times than the other methods as the system dimension increases.

Method Name	BLAZE	Aligator
Success Rate [%]	100	65
Objective Cost	738.7 ± 157.1	635.9 ± 108.81
Solve Time [s]	0.75 ± 0.27	27 ± 11.06
Constraint Violation [%]	2.3 ± 2.5	0.0

TABLE I: Comparison of Success Rate, Objective Cost, Solve Time, and Percent Constraint Violation for Kinova Gen3 trajectory optimization experiments with obstacles. Note that the latter three numbers are just presented for scenarios where both methods were successful. The percent constraint violation represents the average magnitude of constraint violations across all timesteps, computed as the percentage by which state or input constraints exceed their limits at each timestep.

2. We evaluate each of these methods on 20 different scenarios where either the obstacles are randomly placed and x_{init} is generated to be in collision. For 10 of these scenarios, there are 5 obstacles, while for the other 10 there are 10 obstacles. The objective function is formulated to minimize control effort along the trajectory. For each method, we gave a 60s time budget to be able to generate a trajectory. Being unable to generate the trajectory within that time counted as an unsuccessful trial.

As illustrated in Table I, BLAZE is able to generate trajectories much faster than Aligator, and is able to generate trajectories that do not collide with obstacles for all the scenarios. However, in the successful trials Aligator was able to ensure no constraint violation in its forward simulated solution, whereas BLAZE had some slight state constraint violation in its forward simulated solution.

E. Kinova Gen3 Task-Based Scenarios

This section compares BLAZE, Aligator, and RAPTOR under a similar setup as the previous subsection, with two key differences. First, obstacles are now cuboids arranged to resemble more task-oriented scenarios for manipulators that would be deployed in domestic and industrial settings. These scenarios involve actions such as reaching into shelves, retracting arms out of shelves and under shelves, and placing items in confined bins, reflecting real-world manipulation tasks. Second, the time limit for each method is extended to 120s to accommodate scenarios with a greater number of obstacles. For all of these experiments each method must generate a 2-second long trajectory, which is a relatively short time to execute many of these scenarios. In all experiments, each method must generate a 2-second trajectory, a relatively short duration given the complexity of these tasks. This time constraint forces the methods to produce highly dynamic solutions while simultaneously satisfying state and input constraints, as well as obstacle avoidance, making the scenarios particularly challenging. A trial is considered unsuccessful if a method fails to generate a trajectory within the allotted time.

Table II summarizes the performance of the three optimal control algorithms for the 10 different realistic scenarios. In this table "F" denotes that a scenario was considered a failure as per the success criteria introduced in Section VI-B2. BLAZE demonstrates a higher success rate, completing 50% of trials compared to 20% for Aligator and 10% for RAPTOR. Figure 3 illustrates a solution generated by BLAZE for one of the task-based scenarios.

Scenario	Objective Cost	Solve Time [s]	Constraint Violation [%]
1	**723.02**	**0.86 ±0.01**	**2.6**
	F	**F**	**F**
	F	**F**	**F**
2	**F**	**F**	**F**
	F	**F**	**F**
	F	**F**	**F**
3	**773.05**	**2.45 ± 0.02**	**0.0**
	291.35	**57.21 ± 0.13**	**0.0**
	F	**F**	**F**
4	**F**	**F**	**F**
	F	**F**	**F**
	F	**F**	**F**
5	**983.82**	**1.6 ± 0.01**	**3.6**
	F	**F**	**F**
	F	**F**	**F**
6	**F**	**F**	**F**
	F	**F**	**F**
	F	**F**	**F**
7	**F**	**F**	**F**
	F	**F**	**F**
	F	**F**	**F**
8	**F**	**F**	**F**
	F	**F**	**F**
	354.64	**0.323**	**0**
9	**223.78**	**1.28 ± 0.01**	**0.0**
	187.68	**68.46 ± 0.19**	**0**
	F	**F**	**F**
10	**496.95**	**3.1 ± 0.02**	**0.0**
	F	**F**	**F**
	F	**F**	**F**

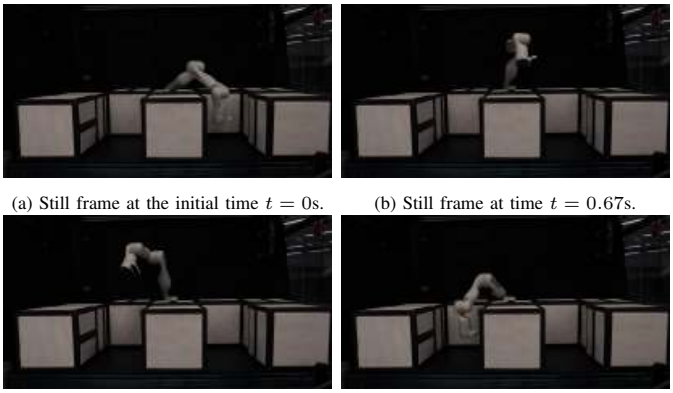
TABLE II: Detailed comparison of Objective Cost, Solve Time and Constraint Violation for the different task-based scenarios. All the problems consider input, state and cuboid obstacle constraints. BLAZE is depicted in **green**, Aligator is depicted in **gold**, and RAPTOR is depicted in **purple**. In this table "F" denotes that a scenario was considered a failure as per the success criteria introduced in Section VI-B2.

Fig. 3: This figure shows a visualization of one of the task-based scenarios (scenario 10). In each subfigure, the color gradient illustrates the evolution in time where the start configuration is in the darkest shade and the end configuration in the lightest. The initial trajectory for the scenario is shown in the top image in yellow and collides with the obstacles along the path. The proposed algorithm is able to push this initial guess out of collision and generate the optimal collision-free solution shown in green in 2.19s for this scenario, whereas the other comparison methods cannot find a solution within the allotted time for this scenario.

F. Hardware Experiments

We demonstrate the performance of BLAZE on hardware on the Kinova Gen3 robot on a number of challenging obstacle avoidance scenarios, where the robot must navigate from some initial position to some final position while adhering to the robot's state and input constraints. Here, the dynamic feasibility of BLAZE as well as its ability to generate trajectories that satisfy the real hardware constraints is critical to enabling the robot to successfully execute the generated trajectory. On the robot, we leverage a passivity-based controller to track the trajectories we generate.

The supplemental material includes videos showcasing various challenging scenarios, such as a rapid pull-and-place maneuver where the robot retracts its arm from a simulated shelf and swiftly places it in a bin (hardware scenario 1, solved in 3.1 seconds); a fast retraction from a confined space while avoiding obstacles (hardware scenario 2, solved in 2.74 seconds); and a precise pick-and-place trajectory requiring navigation through tight grasp points (hardware scenario 3,



(a) Still frame at the initial time $t = 0$s. (b) Still frame at time $t = 0.67$s.

(c) Still frame at time $t = 1.34$s. (d) Still frame at the final time $t = 2.0$s.

Fig. 4: This figure shows a series of still frames of the Kinova arm following the trajectory generated by BLAZE for Scenario 4 in Table II. Each subfigure depicts the arm at a different point in time. The initial guess collides with the obstacle. The proposed algorithm is able to push this initial guess out of collision and generate the optimal collision-free solution in 2.45s, whereas Aligator does it in 57.21s.

solved in 2.45 seconds, as shown in Figure 4). Each of these trajectories is executed within a strict 2-second duration, requiring the robot to move dynamically while adhering to state and input constraints. These experiments highlight BLAZE's ability to rapidly generate dynamically feasible trajectories that fully account for the robot's full-order dynamics, demonstrating its effectiveness in real-world task execution.

VII. Conclusion

This work proposes BLAZE, a generalized AGHF-based trajectory optimization framework that generalizes the AGHF PDE methods to arbitrary cost functions, significantly expanding the range of feasible trajectories. By leveraging the AGHF PDE, this method evolves an initial, potentially infeasible trajectory into a dynamically feasible solution while optimizing a specified objective function, allowing for rapid trajectory generation. A key contribution of this work is the Phase1-Phase2 framework, made possible by the generalized cost function formulation. This Phase1-Phase2 framework addresses a limitation of previous AGHF approaches – requiring constraint-satisfying initial guesses to ensure fast convergence. Phase1 leverages the generalized cost function to drive the trajectory into the feasible set before Phase2 optimizes the objective while maintaining feasibility. This allows the initial guess to violate constraints while ensuring rapid convergence to a valid solution. Additionally, a new method for enforcing input constraints within AGHF is introduced, enabling trajectory optimization with realistic actuation limits without increasing the state dimension, as was required in previous AGHF-based approaches. Simulations and hardware experiments demonstrate that BLAZE can generate trajectories for high-dimensional robotic systems under multiple constraints, with constraint-violating initial guesses faster than state-of-the-art trajectory optimization methods.

VIII. Limitations

BLAZE has its limitations: similar to other AGHF formulations because the action functional (11) incorporates dynamic constraints through a penalty term, the AGHF solution does not minimize the objective function as dramatically as other optimal control methods. In particular, traditional optimal control methods may construct lower cost trajectories; however, in practice, because generated trajectories satisfy all constraints (including input constraints) this may not be too restrictive. In fact if one's objective is to generate trajectories rapidly that are dynamically feasible, BLAZE may be the right approach. One other difficulty with BLAZE stems from its requirement to balance between the constraint penalty, k_{cons}, and the dynamic feasibility penalty, k, which can require some tuning.

References

[1] B. Katz, J. D. Carlo, and S. Kim, "Mini cheetah: A platform for pushing the limits of dynamic quadruped control," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6295–6301.

[2] J. Liu, C. E. Adu, L. Lymburner, V. Kaushik, L. Trang, and R. Vasudevan, "Radius: Risk-aware, real-time, reachability-based motion planning."

[3] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the mit cheetah 3 through convex model-predictive control," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–9.

[4] S. Kuindersma, R. Deits, M. Fallon, A. Valenzuela, H. Dai, F. Permenter, T. Koolen, P. Marion, and R. Tedrake, "Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot," *Autonomous robots*, vol. 40, pp. 429–455, 2016.

[5] J. Michaux, S. Isaacson, C. E. Adu, A. Li, R. K. Swayampakula, P. Ewen, S. Rice, K. A. Skinner, and R. Vasudevan, "Let's make a splan: Risk-aware trajectory optimization in a normalized gaussian splat," *arXiv preprint arXiv:2409.16915*, 2024.

[6] R. Garcia-Rosas, J. M. Portella-Delgado, Y. Tan, and D. Nešić, "Nonlinear observer based control design for an under-actuated compliant robotic hand," in *2016 Australian Control Conference (AuCC)*, 2016, pp. 21–26.

[7] C. Mastalli, R. Budhiraja, W. Merkt, G. Saurel, B. Hammoud, M. Naveau, J. Carpentier, L. Righetti, S. Vijayakumar, and N. Mansard, "Crocoddyl: An efficient and versatile framework for multi-contact optimal control," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 2536–2542.

[8] A. Hereid, O. Harib, R. Hartley, Y. Gong, and J. W. Grizzle, "Rapid trajectory optimization using c-frost with illustration on a cassie-series dynamic walking biped," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 4722–4729.

[9] M. Fevre, P. M. Wensing, and J. P. Schmiedeler, "Rapid bipedal gait optimization in casadi," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, pp. 3672–3678.

[10] S. Liu, Y. Fan, and M.-A. Belabbas, "Affine geometric heat flow and motion planning for dynamic systems," *IFAC-PapersOnLine*, vol. 52, no. 16, pp. 168–173, 2019, 11th IFAC Symposium on Nonlinear Control Systems NOLCOS 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896319317768

[11] C. E. Adu, C. E. R. Chuquiure, B. Zhang, and R. Vasudevan, "Bring the heat: Rapid trajectory optimization with pseudospectral techniques and the affine geometric heat flow equation," 2024. [Online]. Available: https://arxiv.org/abs/2411.12962

[12] L. C. Evans, *Partial differential equations*. American Mathematical Society, 2022, vol. 19.

[13] S. Liu and M. A. Belabbas, "A homotopy method for motion planning," *arXiv preprint arXiv:1901.10094*, 2019.

[14] S. Liu, Y. Fan, and M.-A. Belabbas, "Geometric motion planning for affine control systems with indefinite boundary conditions and free terminal time," *arXiv preprint arXiv:2001.04540*, 2020.

[15] Y. Fan, S. Liu, and M.-A. Belabbas, "Mid-air motion planning of robot using heat flow method with state constraints," *Mechatronics*, vol. 66, p. 102323, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957415820300039

[16] W. E. Schiesser, *The numerical method of lines: integration of partial differential equations*. Elsevier, 2012.

[17] R. Featherstone, *Rigid body dynamics algorithms*. Springer, 2014.

[18] J. Carpentier and N. Mansard, "Analytical Derivatives of Rigid Body Dynamics Algorithms," in *Robotics: Science and Systems (RSS 2018)*, Pittsburgh, United States, Jun. 2018. [Online]. Available: https://laas.hal.science/hal-01790971

[19] E. Polak, *Optimization: algorithms and consistent approximations*. Springer Science & Business Media, 2012, vol. 124.

[20] W. Jallet, A. Bambade, E. Arlaud, S. El-Kazdadi, N. Mansard, and J. Carpentier, "PROXDDP: Proximal Constrained Trajectory Optimization," 2023, https://inria.hal.science/hal-04332348v1.

[21] B. Zhang and R. Vasudevan, "Rapid and robust trajectory optimization for humanoids," 2024. [Online]. Available: https://arxiv.org/abs/2409.00303

[22] L. N. Trefethen, *Approximation Theory and Approximation Practice, Extended Edition*. SIAM, 2019.